P2. Assignment 2.2 OpenSSL and digital signature -Gradescope-

OPENSSL: A Cryptographic Toolkit

Objectives

In this assignment, you will be using OpenSSL library. This is an open-source software widely used because it provides many general purpose cryptographic APIs and it is used to implement Transport Layer Security (TLS) and Secure Sockets Layer (SSL) protocols. You can find more information about ***OpenSSL*** through this link: https://www.openssl.org/

Accessing the CY5001 Master Server

To access the master server, follow these instructions:

- Username: Your first part of Northeastern email address → for e.g, if your e-mail is userid@northeastern.edu, then your username is "**userid**"
- Password: Use the hexadecimal password you received by email.
- Host: cy5001.mscy.org Port: 10010

ssh userid@cy5001.mscy.org -p 10010

Important Notes:

- **The master server is accessed via SSH on a single, common port: 10010, and is the same for all students.**
- **Other assignments may also use the domain cy5001.mscy.org, but those run on different ports assigned individually per student.**
- **Always double-check the port number provided in the assignment instructions.**

Using the wrong port will result in a failed connection or access to the wrong environment.

Part 1

Task 0: Record asccinema cast file

Start an **asciinema recording** and name it **userid.cast,** being your username the one into your NEU email (**userid**@northeastern.edu)

- (i.e.: asciinema rec userid.cast -t "Your Name CY5001 Openssl Assignment".

$ asciinema rec userid.cast -t "Your Name CY5001 Openssl Assignment"

asciinema: recording asciicast to yourname.cast

asciinema: press <ctrl-d> or type "exit" when you're done

$

TASK 1. Symmetric keys

- Use the rand option
- E.g., Generate 128 bit IV (*Initialization Vector*) and key

$ openssl rand -hex 16 > /tmp/iv

$ openssl rand -hex 16 > /tmp/key

$ cat /tmp/iv

53c2ddaa439be09f5a8fef1366ae4579

$ cat /tmp/key

914bc48a7693a8af947120e196f40848

TASK 2. Asymmetric keys

- Use the genrsa and the rsa option to generate a RSA key pair.
- E.g., generate 2048 bit RSA private keys

$ openssl genrsa -out private.key 2048

- E.g., generate public key (saved as userid.pub) from private key

$ openssl rsa -in private.key -pubout -out userid.pub

- Verify that you obtained the proper openSSL RSA keys output. Obviously your keys will be different but double check that they look like these ones.
- $ cat private.key
- -----BEGIN RSA PRIVATE KEY-----
- MIIEogIBAAKCAQEA0Hy+PFrR370V38yo3DXyzO57Qm9br01J6Ck6KEXeeWEJ2Q7z

- BYhrI+bFenkdNkzYQLbjk1N/xMyP6pD2wUpePiBY4gCkfkTRpdVEFHzysSlcSIM0
- 5j9bL9FlDbPbj9STM6SX6x+g16iv6Id5cxOjezAtaQ92DhCNKA4itOh+S7j/TKiJ
- BFMaKVIQer+ge4s+uoyBlBiOSe1W8R+2y8s711YrojTrGsPZFsGNW3EjxxHOQmGR
- IuD2vblaUHvwKMtKC1lri8WBPZ4QjaIMuztyQSdc0ypYfK8IwXbX+3UhIel9sCiw
- xy3DenN2ap7TvHeA/471N0siJhuhdYTrHu4T3wIDAQABAoIBAEK4OPT9p35vd4mY
- aE2Yoiv9Acsf8fpmxIkccHUk7u5vJISnKRSTNJ5Xosjv625OJ0KGhMvIA/KJ+GB7
- pcvzrbc6CNfCluGlymXEmzXnSo9xigvWmV451ke2Ph2lelVMFROIz+k5xPJfFHRY
- roAqP74Ti+S8Qu57U4EHDQpSAODgLcvcpYLCIQ0J3U7ODbFZLConxshkkIcXVFdd
- OOask/zC484yruL5zh7MdxU+IQyYaam4jA9j0pY9FTFP9xSl1qUNrGcoLfdHmzSc
- axh4w9lJNZ9W51qbtAnDeqb9iz2R0b4uzCBhga3Amc0SW1aFonIcbAAlzNYCxMSv
- oYY1mlkCgYEA+vUCQ/TIqDp50paCcjkHW+uvA0VnMDg7D5Q/b5bosRdGP9ISCwgH
- Oq4HGiSdjS4M06m15ZhItHXbyRkdUzeCQMHPbeGHouaSON6GT7OQ3VgPiSS84WHs
- WzQ74N5elcKtWLu81khuhxOLxcbfLDKb1bRB5xVnsBKgITLBP3g+itsCgYEA1K1C
- FtS8YgB7z/qiBaH+VP4nEqvbuU1fxaCNa1/uJFk6Cafykw5xqNaHIwLQPX79gWyR
- L3RIh0stai0XDkHSlYNX88STUomDBIO+koFHMfRYsQu2gr4zS99JPc5X+pfe7ymC
- 9Lx3RIQTCiCHyCSF1fITVLOVb6ld51xbFeIj8E0CgYBLuShA1xjA4jwmFfIwgxK8
- YsZBR2zWePEhR0tI4ZE+9lXlJf75wydjRmWwNsIvemi2aWwQZj3x7eouqNxXYtVz
- ssdjUXz643XVoReH7xfUIiE18ZborwKBlqZGuPJzX/WsY6GcSujG61fuio6GqJKr
- DiQglS3fTA9zVPxIqYC9pwKBgECRjYQS+FKkpPJivHgLU4UIWGKBzMa4aAA/G1Bj
- IpDkigtOcwYUl4tT/lkxzHkvKdX2WSnYGwEAYPxgTiSk9UMxSi2XG8cuOpnPSbd
- CldvQNIXpUSSbE53ruvVzr4lB6kCqGCXK3qgwosLj03DUOLAiP/pOdBZ0kmgr4K4
- UoLtAoGAf/u60vwyCLeaSdGESG8hdL/YXNT3n+F5hbYq3sMR+zzuYMSym/5wshmn
- PqpreY0KDBtCDYgNtW5kKmY89Hst8g4Gl2/6KEiIhVoU+kb3MQkm4314N9IUSN/G
- zLPkFBPXJeLXwA4hn8zNY6CC1Akyw9fxgIliJ6jyx1x8VRYJGw0=
- -----END RSA PRIVATE KEY-----
- $ cat userid.pub

-
- -----BEGIN PUBLIC KEY-----
- MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEA0Hy+PFrR370V38yo3DXy
- zO57Qm9br01J6Ck6KEXeeWEJ2Q7zBYhrI+bFenkdNkzYQLbjk1N/xMyP6pD2wUpe
- PiBY4gCkfkTRpdVEFHzysSlcSIM05j9bL9FlDbPbj9STM6SX6x+g16iv6Id5cxOj
- ezAtaQ92DhCNKA4itOh+S7j/TKiJBFMaKVIQer+ge4s+uoyBlBiOSe1W8R+2y8s7
- 11YrojTrGsPZFsGNW3EjxxHOQmGRIuD2vblaUHvwKMtKC1lri8WBPZ4QjaIMuzty
- QSdc0ypYfK8IwXbX+3UhIel9sCiwxy3DenN2ap7TvHeA/471N0siJhuhdYTrHu4T
- 3wIDAQAB
- -----END PUBLIC KEY-----

TASK 3. Symmetric Encryption

- Use plaintext = "**Your actual name**" and the IV and Key you generated during **TASK 1**
- Using the aes-128-cbc mode and –e option

$ openssl aes-128-cbc -K 914bc48a7693a8af947120e196f40848 -iv 53c2ddaa439be09f5a8fef1366ae4579 -e -in plaintext -out ciphertext

Do not use the same Key and IV that is included in the assignment but the ones you generated in TASK 1

TASK 4. Symmetric Decryption

- Using the aes-128-cbc mode and –d option

$ openssl aes-128-cbc -K 914bc48a7693a8af947120e196f40848 -iv 53c2ddaa439be09f5a8fef1366ae4579 -d -in ciphertext -out decrypted

Do not use the same Key and IV that is included in the assignment but the ones you generated in TASK 1

TASK 5. AES CBC encryption Output

$ cat plaintext

Your actual name

```
$ xxd ciphertext
00000000  29 2b f2 9e 0e 4b e5 09  be 20 76 1d 57 10 b2 b0  |)+...K... v.W...|
00000010  7f 12 bb a1 a4 bb e2 5f  de 85 54 79 b5 a9 6f cc  |....._..Ty..o.|
00000020
```

```
$ cat decrypted
Your actual name
```

```
$ sha1sum plaintext decrypted
515625b0175137c0c396196ccb013b4af34823a1  plaintext
515625b0175137c0c396196ccb013b4af34823a1  decrypted
```

TASK 6. Asymmetric Encryption

- Use plaintext = "**Your actual name**"
- Use the pkeyutl mode and –encrypt option

```
$ openssl pkeyutl -encrypt -inkey public.key -pubin -in plaintext -out ciphertext
```

TASK 7. Asymmetric Decryption

- Use the pkeyutl mode and –decrypt option

```
$ openssl pkeyutl -decrypt -inkey private.key -in ciphertext -out decrypted
```

TASK 8. OpenSSL asymmetric encryption output

```
$ cat plaintext
Your actual name
```

```
$ xxd ciphertext
00000000  7c 8a 03 46 4b 3d 3d d7  37 07 03 c3 89 00 29 e0  ||..FK==.7.....).|
00000010  69 1d dc 9c 38 e4 b6 21  d6 67 11 6a 8a 3f f7 fd  |i...8..!.g.j.?..|
00000020  2f b5 41 66 a2 0d 81 6e  c5 0d 2f 64 58 bb 0d 1e  |/.Af...n../dX...|
```

```
00000030  07 1d 72 1b 4c 48 e1 68  72 7b 70 b2 47 78 cd 1f  |..r.LH.hr{p.Gx..|
00000040  b0 ae c2 2c d9 1a b2 b5  55 49 00 4d 7e 25 ce f1  |...,....UI.M~%..|
00000050  21 e1 d2 ca ee 3b 3c 87  75 2b a2 54 34 ff 2e 4f  |!....;<.u+.T4..O|
00000060  0f dc 7c c6 2f c1 bd b8  24 45 6e ca 53 16 7a 1f  |..|./...$En.S.z.|
00000070  71 9c ef b6 15 e7 a1 27  ae 54 f3 1e 45 d1 91 ee  |q......'.T..E...|
00000080  a8 c3 62 f2 4b ab 76 fb  2a f1 b6 c5 e0 3d 76 0d  |..b.K.v.*....=v.|
00000090  87 13 8c 7a 25 fb d7 47  48 a5 53 73 a8 e4 b2 e4  |...z%..GH.Ss....|
000000a0  e9 39 91 d5 31 72 4a c5  07 98 62 cc 5b 87 ad e1  |.9..1rJ...b.[...|
000000b0  df 83 d1 13 ad 3b 8b 66  34 b5 25 a1 d8 e7 15 5b  |.....;.f4.%....[|
000000c0  27 63 a2 30 e7 82 37 d9  e4 fe 56 1e ce 9f 03 6b  |'c.0..7...V....k|
000000d0  78 35 f6 8f 1d a8 1d 01  73 9a 05 e5 26 66 4b fc  |x5......s...&fK.|
000000e0  53 6b 63 ef a4 79 ef de  0b 70 5f 5c 51 6d d0 42  |Skc..y...p_\Qm.B|
000000f0  bc c0 65 e8 02 bb 88 b0  64 a0 08 a7 14 46 d1 19  |..e.....d....F..|
00000100
```

```
$ cat decrypted
Your actual name
```

TASK 9. Generate md5 hash of plaintext (MD5 is not secure)

- Use plaintext = "**Your actual name**"
- Use dgst parameter for md5 and then filter the output.

```
$ openssl dgst -md5 plaintext | awk {'print $2'}
57afde0e2f9786a7b8061ee833fab3f2
```

`awk { 'print $2' } only shows the second element of the output.

TASK 10. Generate sha256 hash of plaintext

```
$ openssl dgst -sha256 plaintext | awk {'print $2'}
743dfed76fca679dd8b64958bbbb825efa8ca43b81f6e8c1b34e4d311ed80e00
```

TASK 11. Signing with OPENSSL

- Use plaintext = "**Your actual name**"
- Use the dgst mode and -sign option for RSA and SHA256

$ openssl dgst -sha256 -sign private.key -out signature.sig plaintext

TASK 12. Verification

- Use the dgst mode and –verify option

$ openssl dgst -sha256 -verify public.key -signature signature.sig plaintext

Now, stop your asccinema recording and check if your recording is working (asccinema play).

<u>Part 2</u>

Task 1. Digitally sign your cast file

- After completing all previous tasks make sure that you stop the *asciinema recording* and that your userid.cast file is being properly stored into your home directory.
- Generate the digital signature of the userid.cast file that you generated in Part 1 and sign it using **sha-512** digest algorithm with the private key that you also generated during Part 1. Task 2.
- As a result of this task you need to obtain the userid.sig file, also provide in your submission your userid.pub, which contains your public key, that matches the private key you used for the signature.

<u>Part 3</u>

Task 1. Decrypt a Symmetrically encrypted message intended to you

- In this part of the assignment you will be decrypting a symmetrically ciphered message (AES-192 with CTR encryption mode and Salt added) that is stored in the shared folder /home/sharedfolder/proj2.2 of the master server. You need to access to the master server and transfer (SCP) the stored message file, named userid.p22.enc.
- The file has been encrypted using the password you use to connect to the master server.

- Decrypt the file and store the decrypted message into a file named userid.p22.enc.flag

<u>Part 4</u>

Task 1. Find a the original message that generated a MD5 hash

- In this part of the assignment you will be cracking a MD5 hash message ciphered message that is stored in the shared folder /home/sharedfolder/proj2.2 of the master server. You need to access to the master server and transfer (SCP) the stored message file to your system, named userid.MD5-##.
- The file contains two strings:
    1. The original message, where there are several characters that are missing. Characters removed were substituted with X's
    2. The hash of the original message calculated with openssl like this: echo -n "cy5001s25$hex_message" | openssl dgst -md5 | awk '{print $2}'
- You need to bruteforce those characters missing (by substituting the X's) until you find a hash that matches with the second line of the file userid.MD5-##. Then store the complete original message into a file name userid.p22.md5.flag.

$ cat sonrisa.m.p22.md5.flag
aae801bd86d7c33b204e9466bfdfc1b3

>> In order to be able to crack the MD5 faster, we recommend you to use a python program that bruteforce all the potential values of the message. The following library can be very useful to implement a brute force:

>>> for attempt in itertools.product('0123456789abcdef', repeat=7): message = "before"+''.join(attempt)+ "after"; print(message)

>> I recommend creating your own challenge file with a small search space — for example, a low number of brute-force attempts — and where you already know the correct combination. This allows you to quickly test and validate that your cracker script works as intended.

For example, I generated a message using the string 0000fff, so the cracker can find it quickly. Here's an example:

Edit

jose$ echo -n "cy5001s25d59821dd66f047d30000fff95fa689a5" | openssl dgst -md5 | awk '{print $2}'

8b66557c7de44e0fa8cf341350ff0157

jose$ ./easycrack

Original Message: d59821dd66f047d30000fff95fa689a5

MD5 Hash: 8b66557c7de44e0fa8cf341350ff0157

Time taken: 0.01 seconds

Deliverables

Your assignment deliverable files should be submitted in Gradescope. The files to submit are:

Part 1

- userid.cast

Part 2

- userid.pub
- userid.sig

Part 3

- userid.p22.enc.flag

Part 4

- userid.p22.md5.flag

--> IMPORTANT!! Names in files and folder must exactly match the previous specifications

Grading

Please see here the rubric for the assignment.

Concept	**Points**
Part 1: Successful completion of tasks 1-12, shown with .cast file	20
Part 2: Successful digital signature of the cast file	20
Part 3: Successful decryption of your confidential message	30
Part 4: Successful cracking of the MD5 original message	30